

PROJECT REPORT

M E T R I C M I N D

Your Intelligent BI experience

K E S H A V B H A T I

01

**EXECUTIVE
SUMMARY**

02

**PROBLEM
STATEMENT**

03

**SYSTEM ARC.
& TECH STACK**

04

KEY FEATURES

05

**IMPLEMENTATION
DETAILS**

06

**FUTURE VISION &
CONCLUSIONS**

EXECUTIVE SUMMARY

- Metric-Mind is an advanced Agentic Semantic Business Intelligence (BI) Engine.
- It bridges the gap between the flexibility of Generative AI and the reliability of traditional BI dashboards.
- By leveraging a Semantic Layer (Cube-Cloud), Metric-Mind allows businesses users to ask natural language questions about their data and receive trustworthy, governed answers and dynamic visualizations without the risk of AI "Text-to-SQL" hallucinations.
- It also generates interactive graphical visualizations for better Data understanding.

PROBLEM STATEMENT

In modern data analytics, organizations face :

- 1. Traditional BI Dashboards:** Highly reliable and governed, but extremely rigid. Answering new business questions requires engineering resources and long turn-around times.
 - 2. Standard GenAI "Text-to-SQL" Bots:** Highly flexible, allowing users to ask anything. However, they frequently hallucinate SQL logic, leading to inaccurate metrics, breached governance, and an erosion of trust in the data.
- **The Solution:** Metric-Mind solves this by utilizing an Agentic AI workflow that translates natural language not into raw SQL, but into governed API requests to a Semantic Layer.

SYSTEM ARCHITECTURE &

TECH STACK

3.1. Agentic Reasoning

- **Technology:** Google Gemini 3.6 Flash, orchestrated via Lang-Graph.
- **Function:** Acts as the reasoning engine. Instead of directly querying the database, the agent is equipped with specialized tools. It uses React (Reasoning and Acting) logic to determine which metrics to fetch, what dimensions to group by, and what time filters to apply based on the user's intent.

3.2. Semantic Layer

- **Technology:** Cube Cloud (Cube.dev).
- **Function:** Acts as the single source of truth. All business logic (e.g., how "Revenue" or "Active Customers" is calculated) is strictly defined in Cube. The AI requests data from Cube's REST API, ensuring 100% accuracy and consistency with the company's established metric definitions.

3.3. User Experience

- **Technology:** Streamlit and Apache ECharts
- **Function:** Provides a modern, chat-based UI. Users interact with the AI conversationally. When the AI determines that data is best visualized (e.g., time-series data or categorical comparisons), the frontend dynamically intercepts JSON configurations and renders interactive E-Charts directly within the chat flow.

KEY FEATURES

- 1. Governed Natural Language Queries:** Users can ask questions like "What is our total revenue for the last 30 days?" and receive answers derived exclusively from pre-approved metric definitions.
- 2. Zero SQL Hallucinations:** The agent is strictly instructed to never write raw SQL, eliminating the risk of rough database queries and logic errors.
- 3. Dynamic Visualizations:** The agent autonomously determines when a chart is appropriate and generates ECharts JSON configurations, which the Streamlit UI natively renders as interactive graphs.
- 4. Contextual Memory:** Utilizing LangGraph's state management, the agent remembers previous questions in the conversation, allowing for seamless follow-up queries.
- 5. Self-Discovering Metrics:** The agent utilizes a metadata tool to list available metrics in the Semantic Layer, allowing it to adapt to new metrics without requiring code changes.

IMPLEMENTATION DETAILS

5.1. Agent Construction (bi_agent.py)

The core agent is built using LangChain's *create_react_agent* wrapper around the Gemini model. The agent is injected with two primary tools:

- ***list_available_metrics***: Fetches the schema from Cube to understand what can be queried.
- ***query_cube_semantic_layer***: Constructs a JSON pay (Measures, Dimensions, TimeDimensions) and sends it to the Cube API.

5.2. Frontend Integration (app.py)

The Streamlit application manages the chat state and handles message formatting for LangGraph. A critical feature of the frontend is its custom rendering logic:

- It parses the AI's response using Regular Expressions.
- If it detects an echarts_options JSON block, it extracts the JSON, removes it from the visible text and renders the chart using streamlit_echarts

5.3. Configuration & Security

Sensitive credentials (API keys for Google Gemini and Cube Cloud) are managed securely via .env files, ensuring that secrets are not hardcoded into the application logic

FUTURE ENHANCEMENTS

While Metric-Mind currently provides robust semantic querying, future iterations could include:

Multi-turn Data Deep Dives: Allowing the AI to proactively suggest follow-up dimensions (e.g., "Would you like to break this revenue down by region?").

Authentication & Authorization: Tying the Streamlit frontend to corporate SSO, passing user identity down to the Semantic Layer to enforce row-level security.

Export Capabilities: Allowing users to export the generated charts and underlying data to CSV or PDF reports.

BRING YOUR OWN KEY

[BYOK]

As the deployment is based on Gemini API key so it is obvious that the project has only limited resources. So to avoid "Resource Exhausted" error and also to make it open for everyone. I built a BYOK(bring your own key) version in which you can paste your google AI studio API key and run the metric-mind engine.

refer my Project-freetrial folder in github.

CONCLUSION

Metric-Mind successfully demonstrates the next evolution of self-serve Business Intelligence. By marrying the dynamic reasoning of Generative AI with the strict governance of a Semantic Layer, it delivers on the promise of conversational analytics without compromising data integrity.